

Mobiilikehitysprojekti – pisteytys- ja todennuslomake

Tämä lomake palautetaan GitHub-repon /Documents-kansioon. Täytä huolellisesti: se nopeuttaa arviointia ja vähentää väärinkäsityksiä.

Täyttöohje (lyhyesti)

- Merkitse jokaisesta kohdasta: tehty (toimii) tai yritetty (ei täysin toimi).
- Lisää todiste: tiedostonimi/polku + olennaiset koodirivit (alku–loppu).
- Jos kohta on “yritetty”: kirjoita 2–4 lausetta mitä teit ja mikä esti valmistumisen (sekä mahdollinen loki/kuvakaappaus).
- Linkitä videot (loppuvideo ja Advanced-video) tähän lomakkeeseen ja /Documents-kansioon.

1) Projektin perustiedot

Kenttä	Täytettävä arvo
Kurssi / toteutus	Mobiilikehitysprojekti IN00ED17-3003
Tiimin nimi	Ryhmä 13
Jäsenet (nimet)	Andreas, Sara, Mikhail
Teknologia (Android Compose / React Native / muu)	React Native + Firebase
Sovelluksen nimi	TaloFix
GitHub-repo (julkinen URL)	https://github.com/Group13-Sovellusprojekti26/R13-kevat26-Sovellusprojekti
GitHub Projects (URL)	https://github.com/orgs/Group13-Sovellusprojekti26/projects/1
Lyhyt kuvaus ideasta (2–4 riviä)	TaloFix kokoa taloyhtiön vikailmoitukset ja tiedotteet yhteen sovellukseen, jossa asukkaat ilmoittavat, hallitus viestii ja huolto sekä isännöinti hoitavat asiat.

2) Pakolliset palautukset (/Documents)

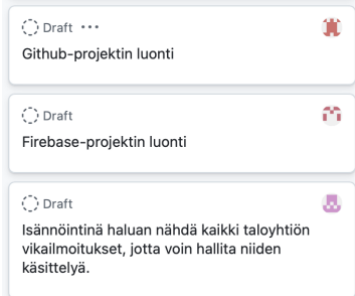
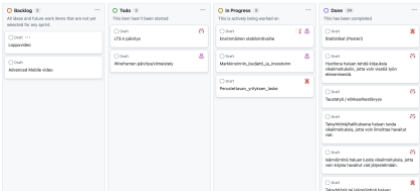
Merkitse tiedostonimi/polku. Jos käytätte linkkejä (YouTube/Stream/OneDrive), lisää myös linkki tähän.

Kohta	Tehty	Tiedostonimi / polku	Linkki / lisätieto
UI-suunnitelma	Tehty	Documents/Wireframe /ui_current/	
Työajanseurantaa	tehty	Documents/Työajan_seuranta.xlsx	

Pisteytyslomake (toteutettu / yritetty)	tehty	Documents/R13_pisteytyslomake_mobiiliprojekti.pdf	
Loppuesitysvideo (≤ 10 min)	Tehty	https://youtu.be/T3lxFsiwC3g	https://youtu.be/T3lxFsiwC3g
Advanced Mobile -video (≤ 12 min)	Tehty	https://youtu.be/qyqiBrYsu-A	https://youtu.be/qyqiBrYsu-A
Scrum muistio	tehty	Documents/R13_scrum_palaverimuistio.pdf	

3) Scrum & GitHub Projects – näyttö

Tarkoitus: arvioija näkee sprintit, tehtävien pilkkomisen, assigneet ja kanbanin etenemisen.

Kohta	Tehty	Todiste (Projects-linkki / sarake / kuvakaappaus)	Huom.
Sprintit nimettynä (Sprint 1–4)	tehty	Backlog · Kevät26 sovellusprojekti-scrum	Sprintit nimetty ja käytetty kurssin aikataulun mukaisesti.
Tehtävät pilkottu (ei liian isoja kokonaisuuksia)	tehty	https://github.com/orgs/Group13-Sovellusprojekti26/projects/1/views/7	Tehtävät jaettu pienempiin kokonaisuuksiin ominaisuuksien mukaan.
Tehtävissä assigneet (kuka tekee)	tehty	GitHub Issues ja Projects – assigneet näkyvissä 	Tehtävät jaettu tiimin jäsenten kesken (Andreas, Sara, Mikhail).
Kanban elää (To do / In progress / Done tms.)	tehty	GitHub Projects – To do / In progress / Done -sarakeet 	Tehtäviä siirretty aktiivisesti sprinttien aikana.
Sprint Planning tehty ennen jokaista Review'ta (lyhyt näyttö/merkintä)	tehty	Tiimin sisäiset sprint planning -keskustelut 	Sprintit suunniteltu ennen Review-tilaisuuksia.
Scrum Review - osallistumiset (päivä + paikalla olleet)	tehty	Kurssin Scrum Review –tilaisuudet Osallistujat (paikalla) Mikhail, Sara, Andreas	Tiimin jäsenet osallistuivat

			Scrum Review - tilaisuuksiin.
--	--	--	-------------------------------

4) Projekti – ominaisuudet (valitkaa yhteensä 18p) + todiste

Täytä jokaiselle valitulle ominaisuudelle: tehty / yritetty, pisteet, ja tarkka kooditodiste (tiedosto + rivit).

Ominaisuus	Pisteet	Tehty	Yritetty	Tiedosto/polku	Rivit (alku–loppu)	Dem o (aikaleima)	Mitä opittiin / mikä jäi kesken (jos yritetty)
HTTP / avoin API + listaus + detali + haku/suodatus + lataus/virhetilat	4	Tehty	☐	faultReports.repo.ts Src/data/repositories/faultReports.repo.ts + faultReports.ts functions/src/faultReports.ts	Rivin alku 354 Rivin loppu: 360	3:14	Opimme tekemään vikailmoitusten listauksen ja tarkemman näkymän Firestoresta sekä toteuttamaan statuksen päivityksen HTTPS-pohjaisen Cloud Function -rajapinnan kautta (httpsCallable, payload: faultReportId + status)
Offline- /välimuistikäytös (näytä viimeisin data ilman nettiä)	2	Tehty	☐	useCompanyFaultReportsVM.ts Src/shared/viewmodels/useCompanyFaultReportsVM.ts (Zustand kirjasto)	Rivin alku 34 Rivin loppu: 73	3:42	Opimme tallentamaan vikailmoitukset paikalliseen välimuistiin AsyncStorage avulla ja näyttämään ne sovelluksessa offline-tilassa. näin käyttäjä näkee viimeisimmät

							tiedot ilman nettiyhteyttä
Sensorit / laiteominaisuus (accelerometer, GPS, askelmittaus, BLE tms.)	4	Te ht y	☐	haptics.ts src/shared/utils/haptics.ts (Käyttöesimerkki: Haptinen tärinä kuvan lisäämisessä ja poistossa src/shared/hooks/useMedia Upload.ts)	1-8 (Käyttöe simerkin koodirivi t: Kuvan lisäys: 83-134 Kuvan poisto: 268- 272)	4:14	Opimme hyödyntämään Expo Haptics - kirjastoa, joka käyttää laitteen tärinää antamaan kevyen, keskitason, onnistumis- ja virhepalautteen käyttäjälle parantaen käyttökokemusta
Taustatyö / elinkaarikestävyys (esim. WorkManager/taust alataus/tila säilyy järkevästi)	3	Te ht y	☐	useAuthVM.ts Src/features/auth/viewmod els/useAuthVM.ts (Käytössä AppProviders.tsx jossa startAuthListener kutsutaan heti sovelluksen käynnistyessä src/app/providers/AppProvi ders.tsx) (Firebase Authentication / Zustand kirjasto)	10-29 (Käyttöe sim: AppProv iders.tsx 20-29)	5:05	ViewModelissa määritellään startAuthListener , joka käynnistää Firebase onAuthStateCha nged-kuuntelijan . Se päivittää Zustand user- ja authReady-kentä t, jolloin aiemmin sisäänkirjautunut käyttäjä säilyy kirjautuneena, vaikka sovellus suljetaan ja avataan uudelleen
Paikallinen tallennus (esim. Room/SQLite/SQLD elight/AsyncStorage) + data säilyy sulkemisen yli	4	Te ht y	☐	useSettingsVM.ts Src/features/settings/viewm odels/useSettingsVM.ts (Zustand kirjasto, tallennus AsyncStorageen)	20-81	5:45	useSettingsVM määrittää Zustand-pohjaise n store:n, jonka loadSettings-fun ktio lukee tallennetun teeman ja kielen AsyncStorage:sta ja päivittää ne tilaan; setTheme ja setLanguage

							tallentavat valinnat AsyncStorageen ja päivittävät i18n-kirjaston, joten käyttäjän teema ja kieli säilyvät sovelluksen sulkemisen ja uudelleenkäynnistyksen jälkeen
Autentikointi (esim. Firebase/Auth0 tms.) + käyttäjäkohtainen näkymä	4	Tehty	☐	RootNavigator.tsx src/app/navigation/RootNavigator.tsx	23–103	6:05	RootNavigator kuuntelee Firebase Authin onAuthStateChanged-tapahtumia. Kun käyttäjä kirjautuu sisään, funktio hakee käyttäjäprofiilin ja tallentaa roolin tilaan. jos profiili puuttuu tai käyttäjä kirjautuu ulos, tilat nollataan. Tämän jälkeen getMainStack() valitsee roolin mukaan oikean navigaatiopinon (AdminStack, HousingCompanyStack, MaintenanceStack, ServiceCompanyStack tai ResidentStack). Näin kirjautumisen jälkeen käyttäjä ohjautuu omaan näkymään ja uloskirjautuneelle näytetään AuthNavigator
Kamera / kuvagalleria	3	Tehty	☐	useMediaUpload.ts	80–197	6:47	Opimme rakentamaan useMediaUpload

(valinta/tallennus/näyttö)				src/shared/hooks/useMediaUpload.ts			hookin, joka hyödyntää Expo ImagePickeria ja DocumentPickeria, jotta käyttäjä voi valita kuvia tai PDF-liitteitä laitteelta. Hook tarkistaa liitetiedostojen koon, tyylin ja lukumäärän, luo uniikin ID:n, muuntaa kuvat base64-muotoon lähettämistä varten ja antaa haptisen palautteen sekä virheilmoitukset onnistumisesta tai virheestä. Tämän ansiosta liitteiden lisääminen vikailmoituksiin ja tiedotteisiin onnistuu luotettavasti.
Vapaa vaativa ominaisuus (opettajan hyväksymä) – kuvaa tarkasti	4	Tehty	☐	useSettingsVM.ts src/features/settings/viewmodels/useSettingsVM.ts Kielituki (fi/en) ja light/dark-teema – vapaa vaativa ominaisuus Perustelut kehityksen vaativuudesta: Kielen vaihtaminen ja teeman hallinta kuulostavat yksinkertaisilta, mutta niiden toteutus koko sovelluksessa vaatii useita osa-alueita: i18n-kirjasto piti konfiguroida ja luoda kattavat	25-81	7:19	Ymmärsimme, kuinka rakentaa useSettingsVM-viewmodeli, joka hallitsee sovelluksen teemaa ja kieltä. Totesimme, että pysyvyyden saavuttaminen edellyttää sekä AsyncStorage-varastoa että lokalisointikirjastoa on hallintaa: piti ladata käyttäjän aiemmat asetukset, vaihtaa i18n-kirjaston kieli ja päivittää Zustand-store,

				JSON-käännöstiedostot kahdelle kielelle; kaikki käyttöliittymän tekstit piti muuttaa dynaamisiksi avaimiksi, jotta käännökset toimivat oikein. Myös teemat edellyttivät oman väripaletin suunnittelua ja React Native Paperin teema-rakenteen laajentamista, sekä integrointia navigaatioon ja komponentteihin. Lisäksi asetusten pysyvä tallennus ja lataus AsyncStoragesta piti toteuttaa, jotta käyttäjän valinnat säilyvät appin sulkemisen yli. Kaiken tämän yhdistäminen toimivaksi, kaksikieliseksi ja kahteen teemaan mukautuvaksi kokonaisuudeksi vaati enemmän työtä ja testausta kuin ensin oletimme.		jotta kaikki komponentit reagoivat muutoksiin. Toiminnot setTheme ja setLanguage päivittävät tilan ja tallentavat valinnat pysyvästi, jolloin teema- ja kieliasetukset säilyvät sovelluksen sulkemisen yli. Tämä opetti meille, että vaikka koodikertymä on suhteellisen pieni, kokonaisuus vaati useiden kirjastojen yhteensovittamista ja testaamista, jotta käyttäjän valinnat toimivat saumattomasti koko sovelluksessa.
--	--	--	--	---	--	---

5) Projekti – UI/UX & navigaatio (7p) – todiste

Kohta	Pisteet	Tehty	Tiedosto/polku tai demo (aikaleima)	Huom.
Näkymät + navigaatio (väh. 3 näkymää, järkevä polku)	2	Tehty	Demo: 0:59	Useita näkymiä ja selkeä navigaatiopolku roolien mukaan
UI johdonmukainen (teema/typografia/komponentit)	2	Tehty	Demo: 1:44	Material Design 3 -teema ja shared UI-komponentit
Käytettävyys: lataus/virhe/tyhjä-tilat, validoinnit	2	Tehty	Demo: 2:02	Lataus- ja virhetilat huomioitu käyttäjäkokemuksessa

Viimeistely: responsiivisuus/landscape/tablet tms.	1	Tehty	Demo: 2:30	Responsiivinen layout mobiililaitteille
---	---	-------	------------	--

6) Projekti – arkkitehtuuri & koodin laatu (7p) – todiste

Suositus: lisää /Documents/architecture.md (5–10 riviä) ja viittaa siihen tässä.

Kohta	Pisteet	Tehty	Todiste (repo-rakenne / tiedosto / rivit)	Huom.
Selkeä kerrosjako (MVVM tai vastaava)	3	Tehty	/Documents/architecture.md esim: src/features/* (Views), src/**/viewmodels/* (VM), src/data/repositories/* (Data) Demo: 7:52	View, ViewModel ja Data - kerrokset erotettu selkeästi.
State-hallinta ja datavirta järkevästi	2	Tehty	/Documents/architecture.md esim: src/app/providers/AppProviders.tsx (auth listener) + src/shared/viewmodels/useCompanyFaultReportsVM.ts (loading/error/refresh) Demo: 8:26	Selkeä ja ennustettava datavirta.
Luettava koodi (nimeäminen, komponentointi, ei turhaa toistoa)	2	Tehty	/Documents/architecture.md esim: src/shared/components/GenericListScreen.tsx + src/shared/components/TfButton.tsx Demo: 8:50	Selkeä nimeäminen, jaettu logiikka ja vältetty toisto

7) Advanced Mobile (3 op) – pisteytys ja todiste (15p)

Aihe on vapaa, mutta se pitää hyväksyttää opettajalla etukäteen. Video max 12 min.

Kohta	Pisteet	Tehty	Todiste (tiedosto/rivit/linkki)	Huom.
Aihe & opetusrunko (mikä/miksi/miten)	3	Tehty	https://youtu.be/qyqiBrYsu-A Aikaleima: 0:00	Video on tehty siitä näkökulmasta, että uudet tekijät jatkavat työtämme. Se briiffaa ja näyttää, miten Firebase backendimme toimii,

				keskittyen yhteen päätoiminnallisuuteen, vikailmoituksiin.
Käyttöönotto & riippuvuudet (kirjastot, konfiguraatiot, permissionit)	4	Tehty	https://youtu.be/qyqiBrYsu-A Aikaleima: 1:52	
Koodidemo (toimiva esimerkki)	3	Tehty	https://youtu.be/qyqiBrYsu-A Aikaleima: 5:03	
Soveltaminen: miten integroitaisiin projektiin (tai demo-projekti)	3	Tehty	https://youtu.be/qyqiBrYsu-A Aikaleima: 8:10	
Videon laatu & hyödyllisyys	2	Tehty	Video on editoitu & äänet kevyesti masteroitu & panostettu, että videon avulla joku voisi jatkaa meidän Firebase ja Talofix projektia ilman meitä.	
BONUS: Advanced-aihe integroituna myös varsinaiseen projektiin	+1	Tehty	Projekti käyttää Firebasea	

8) Bonukset (valinnainen)

Bonus	Pisteet	Tehty	Todiste	Huom.
Projekti: poikkeuksellisen hyvä viimeistely (UX, virhetilat, tyhjätilat, demo)	+1	Tehty	Mielestämme UX-viimeistely on hyvä, ja se näkyy videolla	
Muu opettajan hyväksymä bonus (kirjaa mikä)	0–1	☐		